

CONCIERGE

A multi-tenant AI SaaS. Any business signs up, manages its content in a CMS, and embeds an agent that acts on its public site. The hard part isn't the chat — it's the wall between tenants.

Week 8 Project — Teams of 4

Two teams of 3 collapse the split (see Task Split).
Shared repo. Everyone owns the isolation story and answers for the architecture on Friday.

■ DEADLINE

End of week — submission
Friday — 12-minute team demo

THE MISSION

Build **Concierge**: a SaaS where any business signs up, gets an isolated tenant, manages its website content in a CMS, and embeds an AI **agent** on its public site. The agent doesn't just answer — it **acts**. It retrieves from the tenant's own content, captures leads, and escalates to a human when it's out of its depth. Each tenant configures its agent's persona, enabled tools, and guardrails. The same CMS content powers both the public website and the agent's knowledge.

The hard problem is **isolation**. A visitor chatting on Tenant A must never extract Tenant B's data or your system prompt — even when they try on purpose. Get that wrong and nothing else you built this week matters.

This week's project pulls together what you've built across the program: **agentic AI** (a tool-calling agent, RAG, memory), **software architecture** (layered code, secrets in Vault, blob in MinIO, traces, redaction), and this week's four topics — **System Design for AI**, **CI/CD with GitHub Actions**, **Spec-Driven Development with Skills & Subagents**, and **AI Security, Safety, Guardrails & Compliance**. All of them, in one product.

API-only inference. No local model weights, no torch, no fine-tuning. The LLM and the embeddings are hosted-API calls. This keeps your containers small and your **docker -compose up** fast — and it's the honest stack for an early-stage multi-tenant SaaS. You get the week off from dependency hell. Spend it thinking, not waiting on a build.

Teams of four (two teams of three). One week. Don't add scope — this one is meant to be *educating*, not exhausting.

ARCHITECTURE AT A GLANCE

```
Tenant Admin (Streamlit)          Public site + embedded Widget (React)
|- manage CMS content              \_ visitor chats with the agent
|- configure agent + guardrails    (signed per-widget token, not CORS)
\_ view leads / copy embed snippet |
                                  \_____ one FastAPI backend _____/
                                  |
Tenant Manager (platform) -> provisions / suspends / erases tenants
inbound msg -> CLASSIFIER ROUTER --easy--> workflow handles it directly
                                  \--hard--> tool-calling AGENT picks tools
+-----+
| AGENT -> [ rag | capture_lead | escalate ] bounded loop |
+-----+
scoped by tenant_id . platform rails (locked) + tenant rails (config)
modelserver (onnxruntime / sklearn) -> classifier = the router (NO torch)
pgvector (tenant-filtered) . Postgres . Redis . MinIO . Vault
\_ guardrails sidecar + traces + redacted logs fail CI ___/
```

WHAT MAKES THIS DIFFERENT FROM WEEK 7

- **Many tenants, not one.** Week 7 was a single internal tool. This is a SaaS with many isolated customers sharing one stack. Isolation is the whole game.
- **An agent that acts *and* a model you train.** One LLM picks tools and causes side effects, and it consults a classifier you trained and evaluated yourself.
- **Real ML and DL — trained offline, served lean.** You compare classical ML, a DL model, and an LLM baseline on a real number. No transformer fine-tuning and no torch in any container, so no dependency hell.
- **System design and compliance are graded artifacts**, not footnotes — the scaling story, the cost-per-tenant story, the role model, and the right-to-erasure path.

DESIGN A — SYSTEM DESIGN FOR AI

The graded heart of the week. Written deliverable: `DESIGN.md`.

- **Tenant isolation strategy, decided and defended.** Every row carries a `tenant_id`. The repository layer scopes every query, and the agent's RAG retrieval filters pgvector by `tenant_id` so Tenant A can never pull Tenant B's chunks.

Why — Cross-tenant leakage is the #1 way real multi-tenant AI products fail. Deciding where you enforce isolation — database, app, or vector layer — is the most senior judgment call in this project.

ROLES & TENANT PROVISIONING

- **Three roles, two levels — and no more.** **Tenant Manager** (platform) provisions tenants. **Tenant admin** (per tenant) configures its own agent, widgets, and guardrails and sees its own leads. **Member / visitor** uses the chat. Resist building a configurable permission matrix.

Why — A general RBAC engine is the nightmare. Two levels, three named roles, powers you can count on one hand — that you can reason about, test, and audit.

- **The Tenant Manager is the only role that legitimately crosses the tenant boundary — so it's the most dangerous one.** It provisions, suspends, and triggers erasure of tenants, and reads *aggregate*

cost/usage. It gets **no RLS bypass on content**: it cannot read a tenant's conversations or leads.

Why — “Who is allowed through the wall, and how do you keep that the only gap?” is the real multi-tenant question. The Tenant Manager is a controlled doorway, not god mode — and every action it takes is audit-logged with its actor id.

- **Provisioning and erasure run through a narrow write/delete-only maintenance path**, not a general read bypass. The Tenant Manager can *destroy* a tenant's data without ever being able to *read* it — resolve the “no content access, but must erase” tension explicitly, and log every use of the path.
- **Provisioning flow**: the Tenant Manager creates a tenant and invites its first tenant-admin; the tenant configures itself from there. The platform operator never logs into a tenant to set up that tenant's agent.

Why — Keeps the blast radius small and the privacy line bright: the platform runs the platform, tenants run themselves.

COST, RATE LIMITS & SCALE

- **Per-tenant cost & token attribution**. Every LLM and embedding call is tagged with a tenant. You can answer “what did Tenant X cost us this week.”

Why — An agent that calls paid APIs is a cost center per customer. Not knowing per-tenant cost is how SaaS startups die quietly.

- **Per-tenant rate limiting and a deliberate caching decision**. One noisy tenant can't starve the others. Decide what you cache — embeddings, retrieval results, responses — and what you pointedly don't.

Why — The noisy-neighbor problem only exists under multitenancy. A single-tenant system never has to think about it.

- **A scaling and failure story**. One page in `DESIGN.md`: where this breaks at 10 tenants versus 1,000, and what the next bottleneck is.

Why — Interviews ask exactly this. “It works on my laptop for two tenants” is not a system design.

DESIGN B — ROUTING + ONE AGENT THAT ACTS

The message handler is a **hybrid**, which is what mature LLM apps actually ship: a cheap deterministic **workflow** out front, and one **agent** reserved for the turns the workflow can't resolve. Most real systems are a fixed flow with one bounded agentic step inside — not an agent all the way down.

THE ROUTER (A WORKFLOW)

- **The Design C classifier is the router**. Each inbound message is classified, and a fixed graph handles the enumerable cases directly: spam → drop, a clear FAQ-style question → `rag_search` then answer, an obvious contact/sales intent → `capture_lead`, an explicit “talk to a human” → `escalate`. No LLM reasoning step is spent on cases you can already name.

- **Only ambiguous or multi-step turns reach the agent**. If the router isn't confident, or the turn needs more than one tool in a sequence it can't predict, it hands off to the agent.

Why — This is the production-honest pattern and the cost story: the classifier you trained becomes the orchestration brain, the cheap path carries most traffic, and the expensive agent path is the exception.

Measure it — what fraction of turns you keep off the agent, and what that saves per tenant (feeds Design A).

- **Argue the call in DECISIONS.md**. Make the case for agent vs pure workflow vs this hybrid *for this task*, and defend why the agent earns its slot. Reaching for an agent when a workflow would do is the most

common, most expensive junior mistake — show you didn't make it by accident.

THE AGENT & ITS THREE TOOLS

When a turn does reach the agent, it is a single tool-calling LLM that picks tools under uncertainty — not a second fixed graph.

- **rag_search** — retrieve from the tenant's CMS content and answer.
- **capture_lead** — write a visitor's name, contact, and intent to the tenant's leads table. A real action with a side effect.
- **escalate** — flag the conversation for a human (or open a ticket row) when the agent is out of scope or the visitor asks for a person.

Why — Building and constraining an agent that takes actions is the skill the job market pays for — but only the hard turns need it. Floor: the agent must genuinely handle multi-tool, ambiguous turns, not just sit behind the router as dead weight. On Friday we'll hand the router a turn it should escalate to the agent, and a turn the agent must reason through.

- **Bound the loop.** Cap tool-call iterations and tokens per turn. An agent that picks tools in a loop can *loop* — a hostile visitor who forces long chains drives up that tenant's cost and your bill. The cap is a cost control and a safety control at once.
- **capture_lead is an unauthenticated, LLM-triggered write.** Schema-validate the payload, rate-limit writes per visitor/session, and scope the write to the token's tenant. An injected prompt must not turn it into a spam cannon or a write into another tenant's table.

MEMORY & PROMPTS

- **Short-term session memory in Redis**, scoped per conversation, with an explicit TTL you can justify.

Why — A concierge that forgets the visitor's last message is useless; storing an anonymous visitor's chat forever is a privacy liability. The TTL is where you prove you understand the tradeoff.

- **Prompts live in prompt s/**, version-controlled. Tenant persona is injected at runtime from config — never hardcoded.

Why — Prompts are code. A prompt change with no diff history is an outage you can't bisect.

DESIGN C — YOUR OWN MODEL: ML vs DL, TRAINED OFFLINE, SERVED LEAN

The product is not only LLM calls. You train, evaluate, and ship a real classifier — and you do it without ever putting torch in a container.

- **The task: classify each inbound visitor message by intent** (e.g. sales / support / spam), or score lead quality. The result feeds **capture_lead** (score and triage the lead) and gates the unauthenticated write — spam is dropped before it's stored.

Why — Grounds the ML track in something the product actually uses, and gives the agent a cheap, deterministic signal instead of burning an LLM call on every message.

- **Three approaches, one number.** A classical ML baseline (scikit-learn: TF-IDF + logistic regression or gradient boosting), a small **DL** model, and an **LLM zero-shot** baseline via your API. Compare on a held-out test set — macro-F1, per-class F1, latency, cost — pick one to ship and defend it in **DECISIONS.md**.

Why — “Three models, three numbers, one production” is the real ML-engineering decision. The winner on F1 is not always the winner on latency or cost.

- **Train offline, serve lean — this is how you dodge dependency hell.** Training happens in a notebook / Colab (GPU, torch or sklearn, ephemeral — never in your stack). Export the artifact: the DL model to **ONNX**, the classical model to **joblib**. The **modelserver** container runs only **onnxruntime + scikit-learn + numpy** — no torch, no transformers. The image stays under 500MB and builds in seconds.

Why — Train-heavy / serve-light is the production-honest pattern. The 4GB torch image that broke your Docker builds was never the serving stack — it was the training stack leaking into it. ONNX is how deep learning ships without dragging its training framework along.

- **A model card** with the task, data source + hash, the three results, the deployment choice, and the artifact’s SHA-256. The model-server refuses to boot if the artifact hash doesn’t match the card.
- **Served behind the lean model-server, called over HTTP** with the service credential from Design E. The classifier is a service, not an import.

Dataset: a small public labeled text-classification set (intent or spam) — pick it Monday. Held-out test, no leakage. This is separate from the tenant CMS corpus.

DESIGN D — RAG OVER TENANT CONTENT

Kept deliberately lean. Naive fixed-size chunking + plain dense retrieval is the baseline you beat with **one** justified improvement — not a five-technique stack.

- **Corpus = the tenant’s own CMS content.** Embeddings via a **hosted API** into pgvector, every chunk tagged with `tenant_id`.

Why — API embeddings are why your build is 30 seconds instead of 30 minutes — and why this is the realistic SaaS stack, not a shortcut.

- **One non-naive chunking choice + dense retrieval + one improvement** (a rerank step, a query rewrite, or metadata filtering) — each backed by a number on your golden set.

Why — “Hit-rate went from 0.6 to 0.8 when I switched chunking” is an engineer. “A blog told me to” is not.

- **Retrieval is tenant-filtered at query time.** Part of isolation, not an afterthought.

Why — The most common real-world leak isn’t the database — it’s a vector search that forgot the tenant filter.

DESIGN E — SECURITY, GUARDRAILS & COMPLIANCE

This week’s topic, made concrete. The guardrail you never tried to break is a guardrail you don’t have.

- **Cross-tenant + prompt-injection red-team test, in CI.** A visitor on Tenant A tries to extract Tenant B’s data or reveal the system prompt. The agent must refuse. This test gates merges.

Why — Making it a CI gate is the point — a future refactor can’t silently reopen the hole.

- **Service-to-service calls are authenticated, not just network-adjacent.** API→guardrails sidecar→model endpoints use a shared service credential (or mTLS) resolved from Vault. CORS doesn’t even apply server-to-server.

Why — “It’s on the internal compose network” is not authentication. The sidecar is a trust boundary; an attacker who reaches it shouldn’t be waved through.

- **Two guardrail layers — only one is tenant-editable.** **Platform rails** (prompt-injection, jailbreak, cross-tenant refusal, PII redaction) are mandatory and identical for everyone; a tenant cannot weaken them, and they fail CI when they regress. **Tenant rails** (allowed/blocked topics, refusal tone, persona, enabled tools) are configurable per tenant in the admin page.

Why — A tenant must never be able to dial down injection or cross-tenant defense — that would let one customer turn off the wall that protects every other customer. Business policy is theirs to tune; the security floor is not.

- **PII redaction before anything leaves the service** (logs, traces, memory). A test proves a fake API key pasted into chat never appears unredacted anywhere.

Why — Carries the Week 6/7 standard. Visitors paste secrets into public chat boxes constantly.

- **Right to erasure — a real “delete tenant” path** that purges Postgres rows, pgvector embeddings, MinIO blobs, and Redis sessions. Audit-logged.

Why — GDPR / CCPA erasure is contractual for any SaaS with EU or California customers. “We deleted the row but the embeddings are still searchable” is a compliance failure and a leak.

DESIGN F — THE EMBEDDABLE WIDGET

- **A standalone React widget** (Vite or equivalent), small bundle, served from the API or MinIO with proper cache headers.
- **A loader script at `/widget.js`** — the host pastes one `<script>` tag with their `data-widget-id` and the loader injects the iframe.
- **Theme and greeting come from tenant config at runtime**, read at widget load time.

AUTHENTICATING THE WIDGET — CORS IS NOT ENOUGH

CORS and CSP `frame-ancestors` are **browser-enforced**. They control *where* the widget may be embedded — not *who* may call your API. A `curl` or a script with a copied `widget_id` ignores CORS entirely.

- **The origin allowlist is embedding control, not authentication.** Per-tenant `allowed_origins` in the database drives CORS and the `Content-Security-Policy: frame-ancestors` header — not a hardcoded env var. This stops a browser on a disallowed site; it does nothing to a non-browser caller.
- **The widget authenticates with a short-lived, tenant-scoped token.** The loader exchanges the public `widget_id` (+ allowed origin) for a signed, expiring session token (JWT or HMAC) from the API; every chat request carries it. The **token** is what the API trusts.
- **The verified token also sets the RLS tenant context for the request.** A widget visitor is anonymous — there is no logged-in user to derive a tenant from. The `tenant_id` comes from the *verified* token, never from a client-supplied field. Trusting a `tenant_id` in the request body is a one-line cross-tenant breach.
- **Validate the origin server-side too**, in the request handler — reject a mismatch with a real 403. Treat CORS + CSP as defense-in-depth around the token, never as the boundary itself.

*Why — A CORS error happens in the victim's browser; an attacker doesn't use a browser. If you only set CORS and skip the signed token + server-side check, you bought nothing. **Friday demo:** widget loads on an allowed host, is blocked on a disallowed host (real console), and a raw `curl` with a stale token is rejected by the API.*

SPEC-DRIVEN DEVELOPMENT (HOW YOU BUILD)

This week's "Spec-Driven Development with Skills & Subagents" is *methodology* — and a graded artifact, not just a habit.

- **Write the spec before the code.** Commit a `SPEC.md` per major component. The agent's tool contracts, the isolation rules, the role model, and the eval thresholds are specs you write *first*.
- **Commit the skills and subagents you built** to scaffold the work — e.g. a "tenant-isolation auditor" subagent that greps for unscoped queries, or a skill that generates a new tool from its spec.

Why — The market is moving from "write every line" to "specify, scaffold, review." Showing your specs and subagents proves you can drive an AI coding workflow without losing the thread. **No vibe coding still applies** — you own every line.

CI/CD WITH GITHUB ACTIONS

On every push: lint, type-check, build images, then run the gates below. Thresholds committed in `eval_thresholds.yaml`. Any regression blocks merge.

- The **classifier** eval — macro-F1 on the held-out test set.
- The **agent tool-selection** golden set — did it pick the right tool?
- The **RAG** golden set — retrieval and generation metrics.
- The **injection / cross-tenant** red-team set — every attempt must fail.
- The **redaction** test — the fake key never leaks.
- A **stack smoke test** — the compose stack comes up clean from a fresh clone.

Why — CI that doesn't gate on agent behavior is theater. The point is that the agent can't quietly get worse between Monday and Friday.

RECOMMENDED LIBRARIES (DON'T REINVENT THE WHEEL)

MULTITENANCY & AUTH IN FASTAPI

Use the database to enforce isolation and a known library for identity. Hand-rolling either is how leaks get in.

Auth & roles	fastapi-users — JWT, email/password registration, the role plumbing you already used in Week 7. Layer the Tenant-Manager / tenant-admin / member roles on top. Don't build auth from scratch.
Isolation (core)	Postgres Row-Level Security (RLS) . A <code>tenant_id</code> column, a session variable set per request (<code>SELECT set_config('app.tenant_id', ...)</code>) via a SQLAlchemy event listener or FastAPI dependency, and one RLS <code>POLICY</code> per table. The <i>database</i> refuses cross-tenant rows — not your hand-written filters. Reset the variable at the end of every request — pooled connections persist it, and a leftover value is a cross-tenant leak.
Isolation (depth)	Still scope at the repository layer (<code>.filter(tenant_id == ...)</code>). RLS catches the query a tired developer forgets to scope.
Widget token	PyJWT (or your auth lib's signer) for the short-lived per-widget token, plus a server-side origin check in the handler. CORS/CSP are defense-in-depth, never the auth.
Vectors	pgvector with a <code>tenant_id</code> column on the embeddings table, covered by the same RLS policy — retrieval is tenant-safe by construction.
Optional shortcut	fastapi-tenancy bundles RLS / schema / hybrid strategies. Fine to evaluate, but it's new — understand RLS yourself before you lean on a wrapper.

Why — RLS is the “don't reinvent the wheel” answer for isolation: Postgres has enforced row boundaries for years. A forgotten `.filter()` is then no longer a breach.

THE GUARDRAILS SIDECAR

Run guardrails as a **separate sidecar service** the API calls over HTTP (with a service credential, per Design E). Two real open-source options:

Recommended: NeMo Guardrails	NVIDIA's toolkit for programmable conversation and topic rails — input rails, output rails, jailbreak / prompt-injection detection, keeping a chat inside an allowed domain. Exactly Concierge's need: keep the agent on the tenant's business and refuse cross-tenant / injection attempts. Vendor-neutral on the LLM, runs as a clean sidecar, and mirrors a real production guardrails architecture.
Alternative: Guardrails.ai	Validation-first — a hub of composable validators for output structure and PII . Strong if your main need is shaping/validating outputs rather than topical control.
Pragmatic combo	NeMo for topical + injection rails, plus a Guardrails.ai (or Presidio) validator for PII if you'd rather not hand-write redaction regex. Pick one primary; don't build a platform.

Why — The security centerpiece here is scope control + prompt injection + cross-tenant refusal — squarely NeMo's wheelhouse. Default to NeMo as the sidecar; reach for Guardrails.ai/Presidio only if PII validation is the part you want a library to own.

TRAINING & LEAN MODEL SERVING

Train (offline)	scikit-learn for the classical baseline; PyTorch or Keras for the small DL model — in a notebook or Colab only. This environment is ephemeral and never shipped, so its weight doesn't matter.
Export	DL model → ONNX (<code>torch.onnx.export</code> or <code>tf2onnx</code>); classical model → joblib . Pin the artifact's SHA-256 in the model card.
Serve (lean)	onnxruntime for the DL model, scikit-learn + joblib for the classical one, behind FastAPI in the <code>modelserver</code> container. No torch, no transformers — the image stays small and builds fast.

Why — This is the whole trick to ML/DL without dependency hell: the heavy framework lives in training, the lean runtime lives in serving. They never share a container.

EVALUATION

Four CI gates. Committed thresholds in `eval_thresholds.yaml`.

CLASSIFIER — HELD-OUT TEST SET

- Macro-F1 on the held-out test, gated at a committed threshold. The ML / DL / LLM three-way comparison is committed alongside it — the shipped model can't silently fall behind a baseline it once beat.

AGENT TOOL-SELECTION — 15 EXAMPLES

- Given a visitor message, did the agent pick the right tool, or correctly pick none?

RAG — 15 TRIPLES

- Question / ideal-answer / ground-truth-chunks. Retrieval metrics (hit@k, MRR) and generation metrics (faithfulness, answer relevancy).
- RAGAS or a frozen judge model — your choice. Hand-label a few yourself and report agreement with the judge.

RED-TEAM — THE ATTEMPTS THAT MUST FAIL

- A handful of injection and cross-tenant probes. **All** must be refused for the build to pass.

SUGGESTED 5-DAY SCHEDULE

~6 focused hours/day, split across the team. The agent day is heaviest. Friday is a half day on purpose — take the afternoon to demo and breathe.

MON	Specs & skeleton	Write specs and Claude Code skills/subagents. Compose stack up (reuse Week 6/7 infra), Vault wired, tracing wired, Alembic baseline, tenant model + RLS + role model, Tenant Manager seeds two tenants.
TUE	Models, CMS & RAG	Train the classifier offline (classical + DL→ONNX + LLM baseline), export, stand up the lean model-server. Tenant content + corpus, API embeddings into tenant-filtered pgvector, retrieval with a number.

WED	Router + agent	Classifier-driven router (workflow) for the easy cases; the bounded tool-calling agent for the hard turns. Redis session memory, guardrails sidecar wired (injection + cross-tenant rails).
THU	Widget auth, config, evals, erasure	React widget + loader + signed per-widget token + per-tenant origin allowlist, admin config page, all four eval suites in CI, the delete-tenant path.
FRI AM	Polish + present	Final integration, CI green, READMEs done, practice. Team demo in the afternoon.

SAMPLE TASK SPLIT (TEAM OF 4)

Four owners, four vertical slices. Everyone pairs on the Monday skeleton so the team shares the contracts — then split. Nobody owns a layer in isolation; you own a *slice through* the layers, touching API, service, repo, and infra. Rotate a reviewer on every PR.

OWNER A — Platform, Tenancy, Isolation & Provisioning

Day-1 first move: scaffold the repo + docker - compose with all services, wire Vault, write the tenant_id model with one RLS policy and the three-role model.

- Auth with **fastapi-users**; the Tenant-Manager / tenant-admin / member roles.
- RLS policies + the per-request session-variable dependency; repository-layer scoping.
- The **Tenant Manager** provisioning flow (create tenant, invite first admin) and the audit log.
- Per-tenant cost attribution + rate limiting; the **DESIGN.md** scaling story.

OWNER B — Agent, RAG & Memory

Day-1 first move: stand up a hosted-API LLM call behind a clean app/infra adapter and get one tool-call round-tripping end to end.

- The **classifier-driven router** (workflow) for enumerable cases + the bounded tool-calling agent (rag_search, capture_lead, escalate) for hard turns; the agent-vs-workflow-vs-hybrid argument in **DECISIONS.md**.
- CMS content → API embeddings → tenant-filtered pgvector; chunking + retrieval number.
- Redis short-term memory with a justified TTL; prompts in **prompts/**.
- The agent tool-selection and RAG golden sets; the % routed off the agent.

OWNER C — Models, Security & Guardrails

Day-1 first move: wire tracing from the first commit and stand up the lean model-server + guardrails sidecar shells so the API can call both over HTTP with a service credential.

- The **classifier**: offline training (classical + DL→ONNX + LLM baseline), the model card, and the lean `modelserver` (onnxruntime / sklearn, no torch).
- The **guardrails sidecar** (NeMo) — mandatory platform rails (injection / jailbreak / cross-tenant) plus configurable tenant rails (topics / persona).
- The redaction layer + test, and the cross-tenant / injection red-team set with its CI gate.
- Service-to-service auth (token / mTLS) from Vault across API, sidecar, and model-server.

OWNER D — Widget Auth, Admin UX & CI/CD

Day-1 first move: stand up the GitHub Actions pipeline skeleton so it's green before there's anything to gate, and serve a hello-world widget bundle.

- The React widget, `/widget.js` loader, and the **signed per-widget token** exchange.
- Per-tenant origin allowlist (CSP `frame-ancestors` + CORS) and the server-side origin check.
- The admin Streamlit config page (widgets, guardrail config, embed snippet).
- All four eval gates + smoke test in CI; thresholds in `eval_thresholds.yaml`.

IF YOU'RE A TEAM OF 3

- Drop Owner D: **Owner A** takes the admin config page and the CI pipeline; **Owner B** takes the widget auth and origin allowlist (it's the chat surface B already owns). Owner C's models-plus-security slice is the heaviest — pair on it early, and lean on the classical-ML baseline with a single DL→ONNX export rather than a model zoo.

HOW TO BEGIN, MONDAY MORNING (TOGETHER)

- Pick the open registry of services and write `SPEC.md` for the tenant model, the role model, and the three tool contracts *before* any code. Agree the `tenant_id` convention now — changing it Thursday is agony.
- One person drives the `docker-compose` skeleton on the shared screen while the others write specs and the isolation-auditor subagent. Everyone leaves Monday able to run the stack.
- Define `eval_thresholds.yaml` with placeholder numbers on day one so CI has something to gate from the start — tighten as real numbers land.

RULES

■ ISOLATION IS THE GRADE

A working agent that leaks across tenants scores below a plainer one that holds the wall. The wall is the assignment.

■ CORS IS NOT AUTHENTICATION

The widget authenticates with a signed, short-lived token and a server-side origin check. CORS and CSP are defense-in-depth around it, never the boundary.

■ THE EVALS ARE THE GRADE

Committed thresholds that fail CI when you regress. A polished demo with no working gates scores below a rougher one whose CI is real.

■ EVERY DECISION IS BACKED BY A NUMBER

Chunking, embedding model, retrieval improvement — every choice in `DESIGN.md` is backed by a number on your golden set.

■ LEAN CONTAINERS — NO TORCH

LLM and embeddings are hosted-API calls. Your own classifier is trained offline (notebook / Colab) and served lean — DL via ONNX + onnxruntime, classical via scikit-learn + joblib. No torch or transformers in any container. If any image is over ~500MB, something is wrong.

■ NO VIBE CODING

Spec'd or AI-scaffolded, you own every line. Each teammate will be asked about any part of the system on Friday — not just their slice.

THINK ABOUT

- Your DL model beats the classical baseline by 3 macro-F1 points, but doubles latency and ships a 40MB ONNX artifact. Which one goes to production — and does that answer survive a 10x jump in traffic, or a tighter latency budget?
- Your router keeps most turns off the agent and cuts cost — until it confidently routes a nuanced turn down the cheap path and answers wrong. How do you set the confidence threshold, and which way should it fail: over-escalate to the agent, or risk the cheap miss?
- Where exactly is the tenant filter enforced — and what happens the day a new developer writes a query that forgets it?
- Your widget works and a CORS error blocks a bad origin in the browser. Then someone hits the same API with `curl` from a server, holding a copied `widget_id`. What stops them — and is that thing actually authentication?
- Your Tenant Manager can create and delete any tenant. Should it be able to read one tenant's conversations? Where is that line enforced, and what one code change would quietly move it?
- A visitor pastes their own API key into the chat. Name every place that string could land. How would you know it leaked before they did?
- The injection test passes today. What refactor next month silently reopens the hole, and what stops that refactor from merging?
- "Delete my tenant." Name every place that data lives — rows, vectors, blobs, sessions, traces, logs. Did you get all of them?
- You set the RLS tenant variable per request, but your connections are pooled. A request for Tenant A reuses a connection that still has Tenant B's variable set. Where do you reset it, and how do you prove the reset never gets skipped?
- One tenant sets its guardrails wide open. What can that actually touch — and what can it provably *not* touch, no matter how loose its config?

These are your problems to solve. No hints.

SUBMISSION

Public GitHub repo, tag `v0.1.0-week8`, comes up cleanly with `docker-compose` up from a fresh clone after `cp .env.example .env` and filling in the Vault root token.

Week 8 - Concierge - Team: [names]
Repo: [GitHub URL] Tag: v0.1.0-week8
Tenants seeded: [N] Isolation: RLS + repo-layer + tenant-filtered pgvector
Roles: tenant_manager (platform) | tenant_admin | member - no content RLS bypass
Classifier task: [intent | spam | lead-score] data: [dataset]
Classifier - ML F1=[n] | DL(ONNX) F1=[n] | LLM F1=[n] ships: [choice] - because [one line]
Model served: [ONNX/onnxruntime | sklearn/joblib] artifact SHA-256 pinned in model card
Agent tools: rag_search | capture_lead | escalate
Routing: workflow handled [n]% of turns | agent handled [n]% (cost saved: [one line])
RAG - chunking: [choice] improvement: [choice] hit@5=[n] faithfulness=[n]
Embedding model: [name, hosted API]
Guardrails sidecar: [NeMo | Guardrails.ai] - rails: [input/output/topical/jailbreak]
Widget auth: signed per-widget token + server-side origin check (CORS/CSP = depth)
Service-to-service auth: [service token | mTLS] from Vault
Redis short-term TTL: [n] - because [one line]
Tracing backend: [name] Widget bundle size: [n] KB gzipped
LLM: [provider + model]
Docs: DESIGN.md, SPEC.md, DECISIONS.md, RUNBOOK.md, EVALS.md, SECURITY.md

Ship it. Then take the weekend.